

Appunti di Algoritmi e Complessità

per il corso di Boldi P. @ Università degli studi di Milano

Manuele Ferro

2026 / 2027

Indice

1	Introduzione	2
1.1	Notazione e cenni matematici	2
2	Analisi di algoritmi	4
2.1	Load balancing	4

1 Introduzione

Scopo

L'obiettivo della materia è quello di andare a studiare la complessità degli algoritmi, andare ad approssimare algoritmi che non sono applicabili, e studiare queste approssimazioni.

Aggiungo inoltre che dato che ho appena iniziato a farla, non so niente, e questo paragrafo verrà cambiato nel futuro lei.

1.1 Notazione e cenni matematici

In questa sezione ci saranno un po' di cenni ai concetti che andremo a utilizzare in tutto il corso e le notazioni usate.

Alfabeto

Un insieme finito e non vuoto è detto alfabeto e si indica generalmente con Σ

Insieme delle stringhe

L'insieme delle stringhe di un alfabeto è l'insieme di tutte le possibili sequenze finite su quell'alfabeto e si indica con Σ^*

Numerabilità

Un insieme è detto **numberabile** se questo può essere messo in biiezione con i naturali.

Concatenazione

L'operazione di concatenazione, è un operatore che date due stringhe $s1, s2 \in \Sigma^*$ crea la stringa $s1 \circ s2$, dove $\circ$ concatena le stringhe.

L'operatore di concatenazione è associativa e non commutativa.

Magma, Semigrupp, Monoide

Un magma è una coppia insieme-operazione, in cui l'operazione non è associativa

Un semigrupp è una coppia iniseme-operazione, in cui l'operazione è associativa ($\Sigma^*, \circ$)

Un monoide è un semigrupp in cui l'operazione ha elemento neutro nell'insieme.

Un monoide abeliano è un monoide in cui l'operazione gode anche della proprietà commutativa.

Insieme delle funzioni

L'insieme delle funzioni B^A è l'insieme di tutte le possibili funzioni con dominio in A e codominio in B

$$B^A = \{f\} \quad f : A \rightarrow B$$

Naturali come insiemi

Spesso nel corso capiterà di utilizzare i numeri naturali per andare a indicare il sottoinsieme dei naturali fino a quel numero escluso.

$$0 = \emptyset \quad 1 = \{0\} \quad 2 = \{0, 1\} \quad 3 = \{0, 1, 2\}$$

$$2^* = \{\epsilon, 0, 1, 00, 01, 11, 10, \dots\} \quad 2^A = \{f\} \quad f : A \rightarrow 2$$

$2^{2^*} =$ Famiglia che descrive tutti i linguaggi binari
che non è numerabile, per il teorema di Cantor

$$A^2 = \{(a_1, a_2) : a_1, a_2 \in A\} = \{f : 2 \rightarrow A\} = \{f : \{0, 1\} \rightarrow A\}$$

Problemi di decisione

Dato $L \subseteq \Sigma^*$, un problema di decisione è un problema che dato in input un elemento di Σ^* da in output sì o no, a seconda dell'appartenenza dell'elemento a L

$$x \in \Sigma^* \rightarrow \boxed{P_L} \rightarrow \begin{cases} 1 & x \in L \\ 0 & x \notin L \end{cases}$$

Problema

Un problema π è definito da:

- un insieme di input possibili $I_\pi \subseteq 2^*$
- un insieme di output possibili $O_\pi \subseteq 2^*$
- una funzione $Sol_\pi : I_\pi \rightarrow 2^{O_\pi}$

Rappresentazione degli input

Gli input di un problema come descritti sopra devono essere un'unica stringa binaria. Data questa limitazione, è necessario trovare un modo per andare a codificare più numeri di input in una singola stringa in input. Questo viene fatto tramite alla **gamma di Elias**, che consiste nel codificare i numeri in binario andando a inserire ad ognuno un prefisso composto tanti 1 quanto le cifre del numero, e uno 0 per indicare la fine del prefisso. Questo sicuramente aumenta la lunghezza dell'input, ma lo fa di un valore irrisorio all'ordine di grandezza, quindi trascurabile.

Soluzioni

Le soluzioni sono dettate dagli algoritmi e sono così definite:

$$x \in I_\pi \rightarrow \boxed{A}_\pi \rightarrow y \in O_\pi \wedge y \in Sol_\pi(x)$$

Algoritmo

Un algoritmo è un insieme di istruzioni, noi li vedremo come macchine di Turing.

Macchina di Turing

Una macchina di Turing è una macchina ideale, composta da un nastro infinito con su scritti dati e separatori.

Un puntatore scorre sul nastro avanti e indietro e calcola dai dati scritti una soluzione.

Tesi di Church-Turing

La tesi di Church-Turing dice che la macchina di Turing rappresenta il massimo della potenza computazionale ad oggi raggiungibile. Ogni altro strumento di calcolo, come i moderni linguaggi, è pari ad essa.

2 Analisi di algoritmi

2.1 Load balancing

Input:

Ho in input un numero $n \in \mathbb{N}^+$ di task e un numero $m \in \mathbb{N}^+$ di macchinari.

Ognuna di queste task ha un tempo di esecuzione $t_0, t_1, \dots, t_{n-1} \in \mathbb{N}^+$

Indichiamo con L_i il carico associato a una macchina i .

Indichiamo con L il costo, definito come $L = \max_i L_i$

Soluzioni ammesse:

Le soluzioni ammesse per questo problema sono delle partizioni di n per m :

$$A_0, A_1, \dots, A_{m-1}$$

Il problema è di ricerca del minimo.

Esempio grafico:

Supponiamo di avere 8 task, e 3 macchinari su cui eseguirle.

Tasks:

0	1	2	3	4	5	6	7
3	2	3	1	1	4	5	1

Possiamo trovare una possibile partizione delle task per i macchinari del tipo:

m	tasks	L_i				
	0 1 2 3					
0	<table><tr><td>3</td><td>1</td><td>1</td><td>2</td></tr></table>	3	1	1	2	7
3	1	1	2			
	0 1 2					
1	<table><tr><td>5</td><td>1</td><td>3</td></tr></table>	5	1	3	8	
5	1	3				
	0 1					
2	<table><tr><td>4</td><td>1</td></tr></table>	4	1	5		
4	1					

Vediamo facilmente che il costo L è il massimo tra i 3, ovvero 8. Possiamo allo stesso tempo affermare che non è la soluzione ottima, infatti, è semplice evidenziarne una migliore.

m	tasks	L_i			
	0 1 2				
0	<table><tr><td>5</td><td>1</td><td>1</td></tr></table>	5	1	1	7
5	1	1			
	0 1				
1	<table><tr><td>4</td><td>3</td></tr></table>	4	3	7	
4	3				
	0 1 2				
2	<table><tr><td>3</td><td>2</td><td>1</td></tr></table>	3	2	1	6
3	2	1			

Teorema.

Il problema Load Balancing è NPO Completo

Questo ci porta subito a studiare le soluzioni approssimate.

Inserire algoritmo qui che non ho sbatti, e anche i vari tempi e cazzate varie.

Teorema.

Greedy Load Balancing è 2 approssimato per Load Balancing.

Dimostrazione

Osservazione 1. $L^* \geq \frac{1}{m} \sum_j t_j$

Dim 1. $\sum_i L_i^* = \sum_j t_j$

Per pidgeonholing, non è possibile che $\forall i L_i^* < \frac{1}{m} \sum_j t_j$
quindi

$\exists i L_i^* \geq \frac{1}{m} \sum_j t_j$

Osservazione 2. $L^* \geq \max_j t_j$

Sia $\hat{i}$ la macchina più carica alla fine dell'esecuzione e $\hat{j}$ l'ultima task assegnata a $\hat{i}$.
Allora possiamo dire che:

$$L_{\hat{i}} - t_{\hat{j}} \leq L'_i \quad \forall i \in m \leq L_i$$

A questo punto sommiamo le disequazioni:

$$\begin{aligned} L_{\hat{i}} - t_{\hat{j}} &\leq L'_i \\ m(L_{\hat{i}} - t_{\hat{j}}) &\leq \sum_i L_i \\ L_{\hat{i}} - t_{\hat{j}} &\leq \frac{1}{m} \sum_i L_i = \frac{1}{m} \sum_j t_j \leq L^* \\ L = L_{\hat{i}} &= (L_{\hat{i}} - t_{\hat{j}}) + t_{\hat{j}} \leq L^* + t_{\hat{j}} \leq L^* + L^* \\ L &\leq 2L^* \end{aligned}$$

Corollario. Load Balancing $\in$ 2-APX

Teorema.

$\forall \epsilon > 0$ esiste un input in cui Greedy Load Balancing produce una soluzione di costo:

$$2 - \epsilon \leq \frac{L}{L^*} \leq 2$$

Dimostrazione

Prendiamo $m > \frac{1}{\epsilon}$, e $n = (m - 1) + 1$, con i primi $m - 1$ input pari a 1 e l'ultimo pari a m .

Applicando l'algoritmo su questi input otteniamo:

m	tasks	L_i			
	0 1 2				
0	<table><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	13
1	1	1			
	0 1 2				
1	<table><tr><td>1</td><td>1</td><td>3</td></tr></table>	1	1	3	3
1	1	3			
	0 1				
2	<table><tr><td>1</td><td>1</td></tr></table>	1	1	3	
1	1				
$L = 2m - 1$					

Notiamo invece il caso ottimale su questo input:

m	tasks	L_i			
	0 1 2				
0	<table><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	3
1	1	1			
	0 1 2				
1	<table><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	3
1	1	1			
	0 1				
2	<table><tr><td>1</td><td>3</td></tr></table>	1	3	4	
1	3				
$L = 4$, ottimo					

Piccola digressione

L'input definito è inoltre uno dei pochi casi in cui la media equivale al caso ottimo.

$$\frac{m(m-1) + m}{m} = m - 1 + 1 = \lceil m \rceil$$

$$\frac{L}{L^*} = \frac{2m-1}{m} = 2 - \frac{1}{m} \geq 2 - \epsilon$$

Se ordino Greedy Load Balancing, perdo la sua proprietà di essere ONLINE, ma ne guadagno in approssimazione, infatti:

```

1 Sorto i task in ordine non crescente di durata
2 Applico Greedy Load Balancing

```

Listing 1: Offline Greedy Load Balancing

Teorema.

Offline Greedy Load Balancing è $\frac{3}{2}$ approssimato per Load Balancing.

Dimostrazione

Osservazione 1. Se $n \leq m$, troviamo l'ottimo

Assumiamo quindi $n > m$

Osservazione 2. $L^* \geq 2t_m$ dato che

Dim 2.

$\underbrace{t_0 \geq t_1 \geq \dots \geq t_{m-1}}_{m \text{ task}} \geq t_m \dots$ sempre per pidgeonholing

$\exists i \ L^* \geq L_i^* \geq t_1 + t_2 \geq t_m + t_m$

quindi

$L^* \geq 2t_m$

Sia $\hat{i}$ la macchina più carica alla fine dell'esecuzione.

Possiamo assumere che ad essa siano assegnati almeno due task, altrimenti avremmo l'ottimo.

Sia $\hat{j}$ l'ultimo task assegnato:

$$\hat{j} \geq m$$

quindi

$$t_{\hat{j}} \leq t_m \leq \frac{1}{2}L^*$$

quindi

$$L = L_{\hat{i}} = L_{\hat{i}} - t_{\hat{j}} + t_{\hat{j}} \leq \frac{3}{2}L^*$$

N.B. 1: L'analisi è migliorabile, dimostrando che l'algoritmo è $\frac{4}{3}$ approssimato.
[Graham 1969]

N.B. 2: Load Balancing $\notin$ FPTAS (assunto $P \neq NP$)
[Hochbaum-Shmops 1988]